

Department of Computer Science and Engineering
Individual Contribution Report — Team Assignment

A. Assignment & Team Information

Particular	Details
Course Code & Name	CSA0620 – Design and Analysis of Algorithms
Assignment Title	Efficient Nearest-Neighbour Detection for Air-Traffic Collision Alerts (Divide-and-Conquer)
Team / Group No.	Team 10
Team Members (all names)	SREE B (192524023) VASUDEV REDDY (192424182) ABISHEK K (192521120)
Date of Submission	03/09/2026

B. Student's Own Details

Student Name	Register / Roll No.	Role in the Team
K Abishek	192521120	Divide and Conquer Implementation

C. Task-wise Individual Contribution

S.No	Task / Module Handled	Description of Work Done (in your own words)	Tools / Techniques Used	Approx. Hours Spent
1	Divide-and-Conquer Closest-Pair Implementation	Designed and implemented the Divide-and-Conquer closest-pair-of-points algorithm: sorted the aircraft coordinates by x and y, recursively split the point set into left/right halves, solved each half, and merged the results using a narrow vertical strip check limited to the next 6-7 y-sorted neighbours. Verified the result against a brute-force baseline on the Annexure A dataset and a randomly generated 15-point set, derived the $T(n) = 2T(n/2) + O(n)$ recurrence and solved it using the Master Theorem, and benchmarked running time for n up to 8000 to compare against the brute-force approach..	Python 3 (math, time, random modules), Matplotlib for the runtime chart	25 Hours

D. Team Contribution Percentage Summary

Team Member Name	Register No.	% Contribution	Signature
K Abishek	192521120	40%	K Abishek
Total	Total	100%	

E. Course Outcomes / Skills Applied

CO2: Apply algorithm design paradigms to real-world problems	Applied the Divide-and-Conquer paradigm to the closest-pair-of-points problem for a simulated air-traffic collision check.
CO3 & CO4: Analyze time/space complexity and design efficient algorithms	Derived and solved the recurrence $T(n) = 2T(n/2) + O(n)$ using the Master Theorem to show $O(n \log n)$ time, and compared it against the $O(n^2)$ brute-force baseline.
CO7: Use modern tools for implementation and validation	Used Python for implementation, and Matplotlib with timed benchmarks to empirically validate the theoretical complexity.

F. Challenges Faced and How I Resolved Them

Challenge: In an early version of the merge step, I compared every point in the y-sorted strip against every other point in the strip instead of limiting each comparison to the next few neighbours. This made the merge step effectively $O(n^2)$ again in the worst case (many points sharing a similar x-value near the dividing line), which defeated the purpose of the Divide-and-Conquer approach and showed up as unexpectedly slow runtimes during benchmarking.

Resolution: I revisited the geometric proof that at most 6-7 points in the strip need to be checked per point when the strip is sorted by y-coordinate, and fixed the inner loop to only compare each point against the next 6-7 neighbours in y-order. Re-running the benchmark afterward showed the expected $O(n \log n)$ growth, and the result still matched the brute-force answer on all test cases.

G. Individual Learning Outcome / Reflection

Working through this task helped me understand why the Divide-and-Conquer strategy actually needs its merge step to be linear, not just its split step to be balanced — before this exercise, I under-appreciated how much the $O(n \log n)$ result depends on that strip check being bounded by a constant number of comparisons per point. Debugging the strip-check bug also gave me a much clearer picture of how a small implementation mistake can silently erase the theoretical benefit of an otherwise correct algorithm.

H. Declaration

I declare that the contribution described above accurately represents the work I personally performed for this team assignment, and that I have not misrepresented the work of any other team member as my own.

Student Signature	Date
K Abishek	02/08/2026